SELKIE

TECHNICAL WHITEPAPER

Identity-addressed payments, and the wallet that opens on arrival.

A payment protocol in which the recipient's account is created inside the transaction that funds it — removing the claim step that loses 20–40% of all reward value, and giving software a way to pay humans by name.

```
send 5 CC to @ada
```

Prepared for the BOT Chain Ecosystem Support & Incentive Program

v1.0 · August 2026

1 • Abstract

Selkie is a payments protocol and application in which a social handle is the account. A sender names a recipient the way people already name each other — `@ada` — and if that handle has no account, one is created inside the same atomic transaction that pays it.

This inverts the standard onboarding sequence. Conventionally, a person must acquire a wallet, secure a seed phrase, obtain gas, and surface an address *before* they can receive value. Selkie removes that ordering constraint: value arrives first, and the account exists as a consequence of the payment rather than a precondition of it. There is no claim step, no invite link, and no action required from the recipient at all.

Selkie is live today on the Canton Network DevNet with three production surfaces — an X bot, a Telegram bot, and a web wallet — sharing a single dispatch core. This paper specifies the protocol, documents what is running and verifiable today, and sets out a native deployment plan for BOT Chain, where the combination of EVM compatibility, sub-second blocks, near-zero fees and an AI-agent-centric ecosystem makes the model economically viable for the first time.

2 • The problem

2.1 The claim step is where value leaks

Reward campaigns, airdrops, bounties and community payouts share a failure mode: the recipient must perform technical work to collect. Install a wallet, record twelve words, acquire gas for a chain they have never used, then produce a forty-character address. Industry-wide, between 20% and 40% of distributed reward value is never collected. That loss is not a marketing problem — it is a protocol design problem. The recipient is required to act before the value can exist for them.

2.2 Small payments are economically forbidden on most chains

Social payments are, by nature, small: a $0.50 tip, a $2 bounty, a $5 split. On a chain where a transaction costs more than the value it moves, these payments cannot exist regardless of interface quality. This is the structural reason consumer crypto payments have repeatedly failed to reach scale, and it is a constraint no amount of UX work can remove.

2.3 Software has no way to pay a person

An autonomous agent can call a contract, hold a balance, and settle with another agent. It cannot onboard a human. When an agent needs to pay a person — for a completed task, a labelled dataset, a referral, a piece of work — it encounters a recipient with no address and no wallet, and there is no procedure it can execute to fix that. There is no identity-addressed payment rail between software and people. Agent-to-human payment is therefore not a market with poor tooling; it is a market that cannot presently exist.

THE THESIS

These three problems have a single solution. If accounts are addressed by identity rather than by key material, and are created by the act of payment rather than in advance of it, then the claim step disappears, the recipient needs nothing, and any system able to name a person is able to pay one.

3 • Protocol design

3.1 Handle binding and the uniqueness invariant

A handle maps to exactly one account. This invariant is enforced on-ledger rather than in application code: an `AccountDirectory` contract exposes a consuming choice that serialises registration, and the account is created in the same transaction. Because the choice is consuming, two concurrent registrations for the same handle cannot both succeed — the second sees a spent contract and fails. A rollback therefore leaves no orphaned account, and no reconciliation path is needed.

Handles are namespaced by surface, so an X handle and a Telegram handle are distinct identities that a user may later associate with a single owner. Handle resolution happens before settlement; the ledger never sees a string, only a resolved party.

3.2 Create-on-pay

The core mechanism. When a transfer names a handle with no existing account, account creation and the credit are submitted as a single atomic unit. Either the recipient exists and is funded, or neither occurred. There is no intermediate state in which money is held pending a claim, which means there is nothing to expire, nothing to reclaim, and no unclaimed-balance liability to account for.

This is the property that distinguishes Selkie from link-based or voucher-based "send to anyone" systems. Those systems create a claimable artefact and hope it is redeemed; the unredeemed portion is precisely the 20–40% loss described above. Selkie creates the account instead, so redemption is not an event that can fail to happen.

3.3 Requests are contracts, not messages

Asking someone for money creates a `Request` contract naming the payer. Creating it moves nothing. The payer's approval is a single transaction that creates the transfer and executes it, so there is no window in which funds belong to neither party. The ledger — not the application server — enforces that only the named payer may approve. A request is therefore safe to expose to untrusted surfaces: the worst outcome of a malicious request is that it is ignored.

3.4 Conditional payments

An `Escrow` contract supports two-party conditional payment: funds are debited from the payer in the same atomic transaction that creates the escrow, and are released to the payee, refunded with the payee's consent, or refunded unilaterally after a deadline. This makes agent-to-human task payment expressible as a first-class primitive — an agent can commit funds against a deliverable without either side trusting the other.

3.5 Custody model

Selkie's custody design is stated plainly because it is the first question a technical reviewer asks. On networks where per-user accounts can be provisioned freely, each handle receives its own distinct on-ledger party and holds its own balance. On the shared DevNet node where party allocation is an administrator operation outside our control, deposits are received at a single party with per-handle ownership tracked in contracts — an omnibus model. The BOT Chain deployment uses the per-user model, since account creation on an EVM chain requires no privileged operation.

4 • Contract architecture

Selkie's on-chain surface is deliberately minimal. The complete ledger model is **307 lines across six contracts**. Everything else — command parsing, handle resolution, session management, campaign orchestration, all three user surfaces — lives off ledger and is chain-agnostic.

CONTRACT	RESPONSIBILITY	LINES
Account	Handle-to-party binding, the directory, and the consuming choice that guarantees one handle equals one account	83
Escrow	Two-party conditional payment with release, consented refund and unilateral timeout refund	55
Request	Payment requests as contracts; only the named payer can approve	46
Holding	Per-asset balances owned by a party	44
Transfer	Atomic debit and credit, including create-on-pay for unknown handles	40
Rewards	Bulk campaign payout to a list of handles in one submission	39
Total	Complete on-ledger model	**307**

Line counts are of contract source excluding the test module, verifiable in the public repository at `daml/daml/Selkie/`.

4.1 System architecture

Three surfaces share one core. `bot/src/dispatch.mjs` parses and executes every command, and is called identically by the X worker, the Telegram bot and the web server. Adding a surface — including a machine-facing API for agents — means writing a driver that translates an inbound message into a dispatch call. It does not mean touching wallet logic, and it cannot introduce a divergence in behaviour between surfaces.

5 • What is running today

Every claim in this section is independently verifiable against a public network or the open-source repository. Nothing here is a projection.

81

Active contracts live on DevNet at time of writing, including 16 accounts each at its own party

1.9 cBTC

5.5 cETH

Real tokenised assets held through the CIP-56 token standard, verified live 4 Aug 2026

111

Automated tests against a live participant node, including 12 live integration tests

3

Production surfaces: X bot, Telegram bot, web wallet

5.1 Capability status

CAPABILITY	STATE
Handle as wallet, no setup	Live on Canton 3.5.9
Pay a handle with no account	Live – account created in the same transaction
Request payment from a handle	Live – settles only on the payer's approval
Conditional escrow with timeout refund	Live on the bot surface
Bulk campaign payout	Live over HTTP and Telegram
Inbound deposit from an external wallet	Live – received at a handle-tagged party
Real cBTC / cETH via CIP-56	Settled on the shared DevNet node
X sign-in	Live – OAuth 2.0 PKCE

5.2 Settlement evidence

A read-only query against the shared DevNet participant on **4 August 2026** (ledger offset 511565) returns 81 active contracts at Selkie's operator party: 53 holdings, 16 accounts, 1 account directory, and 4 token-standard transfer records. Real tokenised assets held under CIP-56 at external registrars are **1.9 cBTC** and **5.5 cETH** – accepted through the token standard rather than minted as labels in Selkie's own contracts.

Each of the 16 accounts resolves to a *distinct* Canton party, demonstrating the per-user account model operating on a network Selkie does not administer. One of the four transfer records is outbound – 3.0 cETH sent from the operator party to an individual user's party through the token standard – proving the

complete path from external registrar, into Selkie, and out to a user. Historical acceptance used `TransferInstruction_Accept` with choice contexts from the DA Utility Registry, followed by settlement through `TransferFactory_Transfer`. Ledger update IDs:

> **LEDGER UPDATE IDENTIFIERS**
>
> `1220c66531a3dd8369c94fb6fdfee6b43f22b1c8fc9867f632e8ce77be4a4fbe3c`
> `1220f79f7d18ff74a013f9c7ce11edf34dba1aebe8c0516b82de4be436344aecb3`
> `12205ecfd75f1820753b38f0618f6351f22e10c5d8853f3da0c14c07a76cdf1929`
> `12203c83a812b556fa9dd5159422968b0fa0fbe2c0139f0bd60f437ec732bc0258e6`

Selkie's contract package is deployed to the shared DevNet participant, and the wallet logic runs against that node with configuration changes only — no code changes. Two handles resolve to two distinct parties, one pays the other, and settlement occurs on a network we do not operate. This is the distinction between software that works locally and software that works on a network.

6 • BOT Chain deployment

6.1 Why BOT Chain

Fee economics. Selkie's transaction profile is high-count and low-value — precisely the profile that most chains price out of existence. Sub-second block times and near-zero fees are the first conditions under which handle-to-handle payments of a dollar or less are viable at all. Gas rebates compound this directly: a rebate against a high-frequency micro-payment workload is worth substantially more than the same rebate against a low-frequency high-value one.

Ecosystem thesis. BOT Chain is building settlement infrastructure for autonomous agents. The unresolved gap in that thesis is the human edge: agents can pay each other, but cannot pay people. Selkie supplies exactly that edge, and does so with a mechanism — create-on-pay — that requires nothing from the human at all.

Portability. EVM compatibility means the port is a Solidity implementation of six contracts whose semantics are already specified and proven in production, not an exploratory rebuild.

6.2 What ports, and what is rewritten

LAYER	DISPOSITION
X bot, Telegram bot, web wallet	Ports unchanged — chain-agnostic Node.js and React
Dispatch core and command parser	Ports unchanged
Handle resolution, sessions, campaigns	Ports unchanged
Six ledger contracts (307 lines)	Rewritten in Solidity against the same specification

The create-on-pay invariant maps cleanly to EVM semantics: a registry contract holds the handle-to-account mapping, and the transfer function creates the recipient record and credits it within a single transaction, reverting as a unit on failure. Atomicity, which is the property the whole design depends on, is a native guarantee of EVM transaction execution.

6.3 The agent payment interface

Beyond porting the existing product, the BOT Chain deployment adds a machine-facing endpoint: an agent presents a handle and an amount, and settlement occurs whether or not that handle has ever been seen before. Because this is a driver over the existing dispatch core rather than a parallel implementation, it inherits the identical semantics as the human surfaces — including create-on-pay, escrow and the request flow.

6.4 Growth mechanics

Selkie's distribution is structurally unusual: users arrive as *recipients*, not as signups. A community manager paying 40 contributors produces 40 funded accounts at zero acquisition cost, each already holding a balance. An agent operator settling task rewards produces the same effect continuously. Because every payment on a public surface names its recipient, each settlement is also a public demonstration of the product to an audience that already trusts the sender.

7 • Market

Agent and protocol payouts. Task bounties, data labelling, RLHF work, referral splits and community rewards. In every case the bottleneck is the recipient's missing wallet, and in every case a single call naming a handle removes it.

Emerging-market consumer payments, Nigeria first. The target user does not own a hardware wallet and will not install one. They own a phone and a social handle — which, under Selkie's model, is already a complete account. The same contracts serve an agent paying a worker and a person paying a friend.

8 • Compliance and integrity posture

Selkie's growth metric is funded accounts created by real payments from real senders. This is worth stating precisely in the context of an incentive program, because the mechanism that would inflate such a metric — generating addresses under single control and funding them from one source — is both detectable through funding-source analysis and self-defeating under Sybil-resistance rules that discount common-funded addresses.

Selkie's design gives it no incentive to attempt this. Its distribution advantage comes from third parties paying their own contributors with their own funds; a synthetic user produces no such payment and therefore no growth. We will report only accounts arising from genuine third-party payments, and we accept audit of the funding graph on that basis.

9 • Honest limitations

A whitepaper that lists only strengths is not informative. The following are the real constraints as of August 2026.

- **No mainnet deployment.** Selkie runs on a public test network. Nothing is live on any mainnet, including Canton's.
- **Custody is honest but not final.** On the shared DevNet node, deposits are received at a single party with per-handle ownership tracked in contracts. Per-user accounts are proven on networks that permit self-service account creation, and are the model for BOT Chain, but the omnibus arrangement is what runs on the shared node today.
- **Withdrawal to external wallets is not complete.** The outbound settlement path itself is proven — a cBTC transfer was settled through the token standard — but a general user-facing withdrawal flow is not yet built.
- **Escrow is bot-surface only.** The contract, parser, dispatch path and tests are complete; it is not yet exposed in the web interface.

- **Bot surfaces are single-process.** The X and Telegram workers currently run as one process rather than a redundant deployment. Production hardening of this is part of the deployment plan.
- **Privacy claims are network-specific.** On Canton, payment confidentiality is enforced by the ledger. BOT Chain is a public ledger, so Selkie makes *no* confidentiality claim there. What carries over is the practical property that users exchange handles rather than addresses — not cryptographic privacy, and we do not describe it as such.

10 • References

Live application	selkiepay.vercel.app
Source code	github.com/martinvibes/Selkie
X surface	x.com/SelkiePay
Telegram surface	t.me/selkiepay_bot
Contact	Martin — martinmachiebe21@gmail.com

VERIFY IT DIRECTLY

Reply to **@SelkiePay** on X, or message **@selkiepay_bot** on Telegram, with `send 5 CC to @ada`. No installation, no seed phrase and no test tokens are required to see a wallet come into existence.